

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-ae6e9457aa886a7fd.jsonl`
- SHA-256: `188e1cc1bc65c98aeb0cba6a7ce8b4277e1e6a7fb9fabae4d3536ade3e74bf9b`
- Source modified: `2026-06-13T19:16:48+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:15:39.007Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"test_persisted_candidates_are_superset_of_substrate_windows does not actually prove the superset claim under gating","severity":"med","file":"tests/test_fusion_hybrid.py:79-83","detail":"The test name and the module docstring claim 'the persisted candidate set is the full superset' even when the handoff is gated. But this test runs with default config (no gating), where persistence == handoff trivially, so it proves nothing about the superset-under-gating invariant. The gating test (test_gate_a_drops_low_crossing_window_from_handoff_only) does check len(_stats_calls)==2 while handoff==1, which IS the real superset proof ? so the invariant is covered, but by a different test than the one named for it. The superset test as written would still pass even if persistence were (incorrectly) filtered by the gate, because under default config nothing is filtered.","fix":"Either fold the superset assertion into the gating test (it already has both halves there) or rename this test to reflect that it pins ordering/exactness of persisted spans under no-gating. Optionally add an assert in the gating test that the two persisted spans are exactly the substrate windows, making the superset-under-gating claim explicit."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:42.200Z)

I'll verify this finding by reading the actual test file and the related code.

3. user (2026-06-13T19:15:42.981Z)

```
1      """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3      Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4      keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5      substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
```

```

6     persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7     """
8     from types import SimpleNamespace
9     from unittest.mock import MagicMock, patch
10
11     from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14     # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17              "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19              "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22     def _points():
23         pts = []
24         for f in range(30, 120):          # window A frames: y flips 100<->900 around the net
(~500)
25             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26         for f in range(300, 360):        # window B frames: y == net level -> deadband -> no
crossings
27             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28         return pts
29
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37         return r
38
39
40     def _run(indexing=None, provider_segments=None):
41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43         db = MagicMock()
44         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45         db.add_play_segment.return_value = 200
46
47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
49             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52             patch("os.environ.get", return_value="dummy_key"):
53             preg.get.return_value = provider
54             seg = FusionSegmenter(db, {"indexing": indexing or {}})
55             res = seg.process_video("v1", "clip.mp4")
56             return db, provider, res
57
58
59     def _stats_calls(db):

```

```

60     """Persisted stats dicts, in call order."""
61     return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64     def test_net_crossings_persisted_and_person_off_by_default():
65         db, _provider, _res = _run()
66         stats = _stats_calls(db)
67         assert len(stats) == 2                                # both windows persisted
68         (superset)
69         assert all("net_crossings" in s for s in stats)
70         # Window A (rally) crosses many times; window B (held) zero.
71         assert stats[0]["net_crossings"] >= 2
72         assert stats[1]["net_crossings"] == 0.0
73         # Person cue OFF by default -> NONE of the person features present.
74         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
75         # The 4 base motion keys are still there.
76         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
77                    "track_id_count"} <= set(s)
78                    for s in stats)
79
80     def test_persisted_candidates_are_superset_of_substrate_windows():
81         db, _provider, _res = _run()
82         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
83                  db.add_candidate_segment.call_args_list]
84         assert spans == [(1.0, 4.0), (10.0, 12.0)]            # exactly the substrate windows,
85         unfiltered
86
87     def test_default_hands_off_all_windows():
88         _db, provider, _res = _run()                            # min_crossings default 0 -> no
89         gating
90         assert provider.analyze_video.call_count == 2
91
92     def test_gate_a_drops_low_crossing_window_from_handoff_only():
93         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
94         # Persistence is still the full superset (offline substrate intact)...
95         assert len(_stats_calls(db)) == 2
96         # ...but only the high-crossing window A reaches the AI handoff.
97         assert provider.analyze_video.call_count == 1
98
99     def test_person_cue_adds_features_when_enabled():
100         fake_pd = MagicMock()
101         fake_pd.detections_per_frame.return_value = {
102             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
103             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
104         }
105         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
106                  return_value=fake_pd):
107             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
108             stats = _stats_calls(db)
109             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
110             assert all("net_crossings" in s for s in stats)
111             fake_pd.detections_per_frame.assert_called()        # the person detector WAS used

```

4. assistant (2026-06-13T19:15:45.919Z)

Now let me look at the actual segmenter code to understand the persistence/gating logic, which is central to verifying whether the superset claim is actually proven under gating.

5. user (2026-06-13T19:15:46.650Z)

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13     never imports torch / touches the GPU.
14     - ``_select_for_handoff`` ? optional served Gate A/B: when
15     ``indexing.fusion.min_crossings``
16     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17     are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
20     experiment
21     harness can re-score any variant.
22
23     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24     but
25     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26     seeds
27     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28     default
29     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30     through the
31     experiment harness (EXPERIMENT_HARNESS.md), never silently.
32     """
33     from typing import Any, Dict, List, Optional, Tuple
34
35     from backend.pipeline.segmenters.base import segmenter_registry
36     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37     from backend.pipeline.detectors.base import DetectorRunner
38     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40     from backend.pipeline.detectors import rally_gate
41     from backend.pipeline.detectors import fusion_features as ff
42
43     class FusionSegmenter(TrajectoryHybridSegmenter):
44         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45         B)."""
46
47         family = "fusion"
48         display_label = "Fusion"
49
50         def __init__(self, db: Any, config: Any):
51             super().__init__(db, config)
52             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53             default path).
54             self._person: Optional[Any] = None
55
56         @property
57         def name(self) -> str:
58             return "fusion_hybrid"
59
60         @property
61         def description(self) -> str:
62             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63             rallies, "
64
65             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,

```

```

56         "AI provider sets semantic boundaries.")
57
58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
frame_width: float, video_path: str,
71                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
72         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
73         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
74         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
75         # over the whole trajectory by rally_gate (camera-agnostic).
76         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
77         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
78
79         fcfg = idx_cfg.get("fusion", {})
80         if fcfg.get("cue_flags", {}).get("person", False):
81             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
82         return stats
83
84     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
frame_width: float, video_path: str,
85                         fcfg: Dict[str, Any]) -> Dict[str, float]:
86         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
87         frame range and compute the scale/translation-invariant person features. Lazily
builds
88         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
89         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
90         from backend.pipeline.detectors.person_detector import PersonDetector
91         if self._person is None:
92             self._person = PersonDetector(self.config)
93
94         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
95         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
96         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
97         fw = det["frame_width"] or frame_width
98         fh = det["frame_height"] or 0.0
99
100         poly = fcfg.get("court_polygon")
101         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
102         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
103         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
104         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
court_polygon=polygon, net=net)

```

```

109         def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                                window_stats: List[Dict[str, Any]],
111                                idx_cfg: Dict[str, Any]) -> List[int]:
112             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
128
129
130         segmenter_registry.register(FusionSegmenter)
131

```

6. assistant (2026-06-13T19:15:50.247Z)

Now let me look at the parent `TrajectoryHybridSegmenter` to understand how persistence vs. handoff actually works ? this is the crux of whether the superset claim is meaningful and whether persistence could ever be filtered by the gate.

7. user (2026-06-13T19:15:51.001Z)

```

1         """Shared engine for trajectory-substrate hybrid segmenters.
2
3         tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4         standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5         now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10        A concrete subclass supplies only its identity and runner wiring:
11        - ``family`` ? config section under ``indexing.*``, output subdir, and
12          the ``<family>-hybrid-<model>`` detector tag in metadata.
13        - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14        - ``name`` / ``description`` properties (the registry contract).
15        - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16          detection_params extras such as weights_path).
17
18        This base is deliberately NOT registered; only concrete subclasses register.
19        """
20        import os
21        import time
22        import logging
23        import concurrent.futures
24        from typing import Any, Dict, List, Optional, Tuple
25
26        import cv2
27
28        from backend.pipeline.segmenters.base import BasePlaySegmenter
29        from backend.utils.video import get_video_duration, extract_video_chunk
30        from backend.sports import sport_registry
31        from backend.providers import provider_registry

```

```

32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84         -----
85         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86         frame_width: float, video_path: str,
87         idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89             BASE
90             returns exactly the 4 motion keys, so the trajectory twins persist byte-
91             identical
92             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
93             features). ``points`` are the whole-video trajectory points
94             (TrajectoryPoint)."""
95             return {
96                 "peak_velocity": cw["peak_velocity"],

```

```

91         "mean_velocity": cw["mean_velocity"],
92         "active_frame_count": cw["active_frame_count"],
93         "track_id_count": cw["track_id_count"],
94     }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                             window_stats: List[Dict[str, Any]],
98                             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                      player_pool: Optional[List[str]] = None,
108                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)

```



```

152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,
160             "merge_gap": merge_gap,
161             "min_action_window_duration": min_window_duration,
162             **runner_params,
163         }
164
165         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166         ok, msg = runner.healthcheck()
167         if not ok:
168             logger.error("%s WSL environment not ready: %s", label, msg)
169             return []
170         logger.info("%s WSL env OK: %s", label, msg)
171
172         # --- Phase 2: trajectory -> candidate rally windows ---
173         # Trajectory detectors process the whole video in one pass (they carry
174         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175         t_start = time.time()
176         out_dir = os.path.join("output", self.family)
177         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178         if not csv_path:
179             logger.error("%s produced no trajectory; aborting.", label)
180             return []
181
182         points = runner.parse_trajectory_csv(csv_path)
183         logger.info("Parsed %d trajectory points (%d visible).",
184                     len(points), sum(1 for p in points if p.visible))
185
186         all_candidate_windows = runner.trajectory_to_action_windows(
187             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189             min_window_duration=min_window_duration, chunk_index=0,
190         )
191         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                     label, time.time() - t_start, len(all_candidate_windows))
193
194         if log_candidates:
195             for cw in all_candidate_windows:
196                 logger.info(f"[{label}] rally
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198                             f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200                             f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204         fusion
205         # segmenter adds net_crossings + person-cue features. Computed once, reused for
206         both
207         # persistence and the handoff gate below.
208         window_stats = [
209             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210             idx_cfg)
211             for cw in all_candidate_windows
212         ]
213
214         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216         if store_candidates:

```

```

211         for i, cw in enumerate(all_candidate_windows):
212             candidate_ids[i] = self.db.add_candidate_segment(
213                 video_id, detector=self.name,
214                 start_time=cw["start"], end_time=cw["end"],
215                 chunk_index=cw["chunk_index"], confidence=min(1.0,
216                     cw["peak_velocity"]),
217                 stats=window_stats[i], detection_params=detection_params,
218             )
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
224     # candidates above stay the full superset ? only the served play_segments are
225     # gated.
226     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
227         idx_cfg)
228     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
229     os.makedirs(handoff_dir, exist_ok=True)
230     logger.info("Phase 3: %s validation on %d candidate windows...",
231         provider_name, len(handoff_indices))
232
233     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
234         w_start = max(0, window["start"] - handoff_padding)
235         w_end = min(video_duration, window["end"] + handoff_padding)
236         w_dur = w_end - w_start
237         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
238         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
239             reencode=True):
240             return []
241         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
242         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
243             label=f"Handoff {wi+1}")
244         valid = []
245         for seg in segments:
246             try:
247                 seg["start_time"] = float(seg["start_time"]) + w_start
248                 seg["end_time"] = float(seg["end_time"]) + w_start
249                 seg["_candidate_id"] = candidate_ids[wi]
250                 valid.append(seg)
251             except (ValueError, TypeError, KeyError) as e:
252                 logger.warning("Dropping segment with unparseable timestamps: %s ?
253 %s", seg, e)
254         try:
255             os.remove(w_path)
256         except OSError:
257             pass
258         return valid
259
260     ai_segments: List[dict] = []
261     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
262     executor:
263         futures = [executor.submit(process_candidate_window, wi,
264             all_candidate_windows[wi])
265             for wi in handoff_indices]
266         for future in futures:
267             ai_segments.extend(future.result())
268     try:
269         os.rmdir(handoff_dir)
270     except OSError:
271         pass
272     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
273         len(ai_segments))

```

```

267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])
274             dur = end - start
275             metadata = {
276                 "shot_count": self._shot_count_from(segment, dur),
277                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                 "detector": f"{self.family}-hybrid-{model_name}",
279             }
280             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281             if segment_id:
282                 candidate_id = segment.get("_candidate_id")
283                 if candidate_id is not None:
284                     self.db.link_candidate_to_segment(candidate_id, segment_id)
285                 segments_data.append({
286                     "id": segment_id, "start_time": start, "end_time": end,
287                     "duration": dur, "metadata": metadata,
288                 })
289             except Exception as e:
290                 logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

8. assistant (2026-06-13T19:16:29.000Z)

Now I have the full picture. Let me carefully analyze the finding's claims against the code.

The key architecture:

- `window\_stats` is computed for ALL windows (line 203-206)
- Persistence (lines 209-217) iterates over `all\_candidate\_windows` unconditionally ? it persists ALL windows, never filtered by gate
- `\_select\_for\_handoff` (line 225) returns `handoff\_indices`, which only affects Phase 3 (the AI handoff)

So persistence and gating are **structurally decoupled** ? persistence loops over `all\_candidate\_windows`, the gate only filters `handoff\_indices`.

Now let me carefully evaluate the finding's central claims:

Finding claim 1: "this test runs with default config (no gating), where persistence == handoff trivially, so it proves nothing about the superset-under-gating invariant."

Let me verify: `test\_persisted\_candidates\_are\_superset\_of\_substrate\_windows` calls `\_run()` with no args, so `min\_crossings` defaults to 0 ? no gating. The test asserts `spans == [(1.0, 4.0), (10.0, 12.0)]`. Indeed, under default config, nothing is gated. So this test asserts that the persisted spans exactly equal the two substrate windows. This proves persistence == substrate under no gating, but does NOT prove persistence is a superset when gating drops a window.

Finding claim 2: "The gating test ... does check len(_stats_calls)==2 while handoff==1, which IS the real superset proof."

Let me verify `test\_gate\_a\_drops\_low\_crossing\_window\_from\_handoff\_only` (lines 90-95): with `min\_crossings=2`, it asserts `len(\_stats\_calls(db)) == 2` AND `provider.analyze\_video.call\_count == 1`. So 2 windows persisted while only 1 handed off ? this IS the actual proof that persistence remains a superset even when the gate drops one window from handoff. Confirmed.

Finding claim 3: "The superset test as written would still pass even if persistence were

(incorrectly) filtered by the gate, because under default config nothing is filtered."

Let me verify this carefully. Under default config (``min_crossings=0``), ``_select_for_handoff`` keeps all windows. Now, if persistence were hypothetically (incorrectly) coded to filter by the gate, then under default config the gate keeps everything anyway, so persistence would still produce both spans, and the superset test would still pass. So the test cannot catch a regression where persistence becomes gate-coupled. This is a true observation about test sensitivity.

Now, the critical adversarial question: ****Is the finding fairly characterizing the test naming/coverage, and is this a "real issue"?****

Let me check the test name and docstring claim precisely.

The module docstring (lines 1-7) says: "the persisted candidate set is the full superset of the substrate windows; served Gate A drops sub-threshold windows from the AI handoff while persistence keeps the superset".

So the docstring actually splits these into TWO clauses:

1. "the persisted candidate set is the full superset of the substrate windows"
2. "served Gate A drops sub-threshold windows from the AI handoff while persistence keeps the superset"

Let me re-read the finding's claim: "The test name and the module docstring claim 'the persisted candidate set is the full superset' even when the handoff is gated."

This is where I need to be adversarial. The docstring clause "the persisted candidate set is the full superset of the substrate windows" does NOT actually say "even when the handoff is gated." The "even when gated" semantics is in clause 2 ("while persistence keeps the superset"), which maps to the gating test. So the finding slightly mischaracterizes the docstring by injecting "even when the handoff is gated" into a clause that maps to the no-gating test.

The test name is ``test_persisted_candidates_are_superset_of_substrate_windows``. "Superset of substrate windows" ? under no gating, the persisted set equals the substrate windows (which is trivially a superset, the improper case). The test asserts exact equality ``spans == [(1.0, 4.0), (10.0, 12.0)]``, which is the substrate. So the test name is technically accurate: persisted candidates ARE a superset (in fact, equal) of substrate windows. The word "superset" here is about the persistence-vs-substrate relationship, not persistence-vs-handoff.

Let me reconsider. The finding's core technical observations are accurate:

1. The named "superset" test runs without gating ?
2. It would still pass if persistence were gate-coupled ? (because gate is a no-op at default)
3. The actual superset-under-gating invariant is proven by the gating test ?

But is this a "real issue" or a nitpick? Let me weigh:

- The invariant IS covered (by the gating test, line 93). The finding explicitly concedes this: "so the invariant is covered."
- The criticism is essentially about test **naming/organization** ? the test named "superset" doesn't prove the superset-**under-gating** property, but a different test does.
- The "superset of substrate windows" test name is arguably accurate for what it checks (persisted == substrate, exactly).

Let me check whether the docstring genuinely overclaims for THIS test. The docstring is a module-level docstring covering ALL tests, not a per-test claim. The "superset" word appears in two distinct contexts in the docstring, and the gating clause ("while persistence keeps the superset") correctly maps to the gating test.

Let me verify there isn't a per-test docstring on the superset test that overclaims.

Looking at lines 79-83: ``test_persisted_candidates_are_superset_of_substrate_windows`` has NO docstring. So there's no per-test overclaim. The comment on line 82 says ``# exactly the substrate windows, unfiltered`` ? which is accurate.

Let me now assess severity and whether to confirm.

The finding labels this "med". Let me scrutinize that. The finding itself concedes "the invariant is covered, but by a different test than the one named for it." So this is fundamentally a test-organization/naming critique, not a coverage gap or a bug. The actual gating-superset invariant IS tested (line 93: ``assert len(_stats_calls(db)) == 2`` with `handoff==1`).

Is the test "not proving the superset claim under gating"? The test name doesn't say "under gating" ? it says "superset of substrate windows." Under no-gating, `persisted == substrate`, which is a valid (if improper) superset, and the test pins that exactly. The finding is reading "under gating" into a test that never claimed it.

Now, is there genuine value here? There's a mild point that the test suite's naming could be clearer, and the suggestion to add an explicit "the two persisted spans are exactly the substrate windows" assertion to the gating test would strengthen it. But the gating test already asserts ``len(_stats_calls(db)) == 2``. Adding span-exactness would be marginally stronger but isn't a real gap ? the existing two tests together cover: (a) `persisted spans == substrate` exactly (no-gating test), and (b) `persisted count stays 2 while handoff drops to 1` (gating test). Together these DO establish the superset-under-gating invariant for this scenario.

Let me double-check one subtle thing: does the gating test verify that the **same two windows** are persisted, or just that **two** are persisted? Line 93 only checks ``len(...)`` (count), not the spans. So technically the gating test proves "2 persisted, 1 handed off" but not "the persisted ones are exactly [(1,4),(10,12)]." However, combined with the no-gating test asserting the exact spans, and given persistence is structurally gate-independent (it loops ``all_candidate_windows`` regardless of ``min_crossings``), the coverage is adequate.

This is a defensible but minor test-quality observation. It is NOT a bug, NOT a missing-coverage gap (the invariant is covered), and the "med" severity is overstated. The finding even mischaracterizes the docstring slightly by attributing "even when gated" to a clause that maps to the no-gating test.

Verdict: This is essentially a nitpick about test naming/organization. The invariant the finding worries about is already covered by the sibling gating test (which the finding admits). The test as named accurately checks "`persisted == substrate windows, unfiltered.`" The default-`to-real=false` instruction applies to nitpicks/test-organization quibbles where the actual behavior is correctly covered.

I'll mark this as not a real issue (it's a low-value naming/organization nitpick, and the invariant is genuinely covered elsewhere), but I'll capture the small kernel of truth in the reasoning.

9. user (2026-06-13T19:16:41.596Z)

Structured output provided successfully

10. assistant (2026-06-13T19:16:48.713Z)

I verified this finding against the actual code and concluded `**real=false**` (low-value nitpick).

The finding's three technical observations are literally accurate, but it does not constitute a real defect:

- `**The invariant is genuinely covered**` ? the finding itself concedes this.
``test_gate_a_drops_low_crossing_window_from_handoff_only`` (tests/test_fusion_hybrid.py:93,95) proves the superset-under-gating property by asserting 2 persisted vs 1 handed off.
- `**Persistence is structurally gate-independent**` ? in trajectory_hybrid.py:209-217 the persistence loop iterates ``all_candidate_windows`` unconditionally; only ``handoff_indices`` (line 225) is gated. The named test accurately checks "`persisted == substrate windows, unfiltered`" under default config.
- `**The docstring is not overclaiming for that test**` ? the module docstring's gated clause ("`while persistence keeps the superset`") maps to the gating test, not the named superset test, which the finding slightly mischaracterizes.

The criticism reduces to "the invariant is proven by a sibling test rather than the one whose

name says 'superset'" ? a test-organization/naming quibble, not a bug or coverage gap. The only kernel worth keeping is the optional suggestion to add a span-exactness assertion to the gating test (it currently checks count==2, not the specific spans), which I noted in the fix field as polish.